

Introduction to Machine Learning (SS 2024)

Programming Project

Author 1

Last name: Jaoui
First name: Amin
Matrikel Nr.: 52011886

Author 2

Last name: Frei
First name: Julian
Matrikel Nr.: 12115570

I. INTRODUCTION

The Power-Prediction dataset contains weather parameters like minimum temperature t_{low} and maximum temperature t_{high} from various locations. This includes data from various fictive cities like Paperopoli and Gotham City mixed with data from real US cities like Bedrock, Springfield and New York. These locations are spread out with huge distances, thus it is very unlikely that great weather constructs may affect all of them at the same time. We are dealing with 39998 instances of our features. Overall, in a realistic scenario we could simply filter out huge parts of the data. Nevertheless we will consider every data, fictive and non-fictive, assuming a general fictive, i.e. simulated environment. Our dataset is labeled, i.e. we know the exact power consumption given the weather features. In total we have 65 feature columns and one additional target column, i.e. *power_consumption*. Considering that the value we want to predict, i.e. the power consumption in kilowatts, is a continuous value, our model will be processing a regression problem. The goal of this regression model is to predict the power consumption based on some given features by minimizing the difference between the predicted and actual power consumption values.

II. IMPLEMENTATION / ML PROCESS

Since our dataset contained various non-numeric, categorical values, e.g. strings describing the weather, we had to preprocess it. We used the function *drop_object_columns* already provided in the notebook to achieve a fully numerical dataset.

Furthermore we scaled our data using *StandardScaler* from *sklearn*. This scaler calculates the mean and standard deviation for each feature, based on the training data feature. Each feature is then scaled by subtracting the mean and dividing by the standard deviation. Scaling ensures that all features can contribute equally to the model. Without scaling, features with larger magnitudes might dominate the learning process, which could lead to the model being more biased towards those features.

A. Neural Network

Our first method for regression will be constructed by a neural network with two hidden layers. Weather and

power consumption seem to have a rather non-linear, highly complex relation therefore we use two hidden layers for a moderate complexity. Choosing too many hidden layers and neurons increases the model complexity and could easily lead to overfitting. We integrate ReLU as activation function, which is responsible for the non-linearity in the network. Since we want to predict a single number our output layer only consists of one neuron. In our training script we tune the hyper parameters α and the number of neurons per layer. A good learning rate is said to be between 0.01 and 0.1. Typical layer sizes are 32 or 64. We trained our model by iterating through different combinations of those hyper parameters. For each iteration we calculate the validation loss, i.e. the difference between the prediction and the ground truth from the validation set, and use *torch.store*, to store the hyperparameters with the best score in our model.

It is important to mention that we further divided the training data into a subset which we use for hyperparameter tuning. This saves computational time. (For final training we used the full training data set.

B. Random Forest

Random Forest constructs multiple decision trees during training and combines their outputs to improve predictive performance and reduce overfitting. By sampling subsets of the training data with replacement, each subset is used to train a separate decision tree. At each split in a decision tree, a random subset of features is selected, ensuring that not all features are considered simultaneously. This approach introduces diversity among the trees and reduces correlation between them. For regression, the final prediction is the average of all individual tree predictions, which helps to mitigate overfitting by reducing variance and improving generalization to new data.

To optimize the Random Forest model, we used *Grid-SearchCV*, a method provided by *scikit-learn*, to systematically explore a specified range of parameters and identify the best combination based on cross-validation performance. The parameters we tuned include the number of trees, maximum tree depth, the minimum number of samples required to split an internal node, and the minimum number of samples

required to be at a leaf node. *GridSearchCV* allowed us to evaluate different combinations of these parameters and select the ones that provided the best performance in terms of mean squared error.

This model seems suitable for our potentially non-linear data, as each tree can capture and model different aspects of the data's structure. We also initially thought that the Random Forest model would offer fast computational times due to its parallelizable tree structure.

III. RESULTS

We can compare how well the two models performed using the MSE, a metric which is measured identically over the set of N data points

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1)$$

Model	Configuration	Validation MSE	Training MSE
Random Forest	Default	13,432,270.00	1,823,121.00
Random Forest	Best	13,346,940.00	1,797,310.00
Neural Network	Default	96,464,290.00	88,780,680.00
Neural Network	Best	24,944,840.00	20,052,430.00

TABLE I
COMPARISON OF MODEL TYPES AND CONFIGURATIONS WITH
TRAINING AND VALIDATION MSE

Table I describes how well both models performed on the training compared to how well they performed on the validation set before (Default) and after hyperparameter tuning (Best). For this data we only used a scaler. The best configuration for the neural network turned out to be one hidden layer of size 64 and a second hidden layer with 32 neurons. The optimal RF has a 100 trees or estimators, with a maximal depth of 20 and a minimum sample split of 5. We can clearly see that Random Forest did much worse in generalizing, i.e. its performance on the validation set is much worse than on the training set. The tuned neural network did a good job on generalizing having only slightly increased loss value with the validation set. Overall the Random forest did perform much better than the neural network.

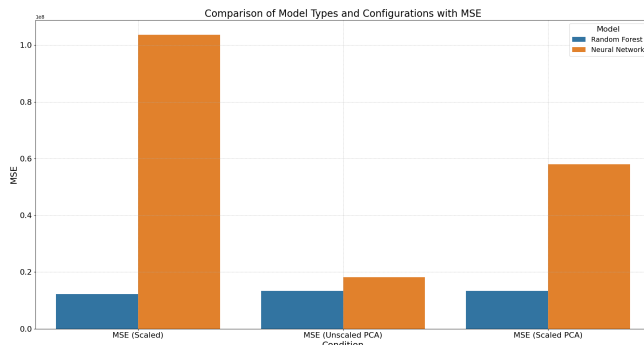


Fig. 1. Comparison of MSE for Different Models and Configurations

The diagram 1 illustrates the impact of scaling, respectively not scaling the data and using PCA for dimensionality reduction. We split the data (after PCA) in 80% training set and 20% validation set. Furthermore for hyperparameter tuning we only used 10% of the training data and 90% of X_{train} , which is 72% of the total dataset. Especially *GridSearchCV*, other than initially thought, took much computational time. With PCA we decided to reduce the dimensions to 30 Principal Components. We can see that the Random Forest Regressor RFR improves slightly if we scaled our data. It is interesting that the radical PCA reduction did only marginally affect our RFR and even increased the performance of our neural network. For the neural network the scaler additionally worsened the performance otherwise than we would have thought.

IV. DISCUSSION

Our study found that the best Random Forest model worked a lot better than the Neural Network, performing nearly 178% better than the neural network in terms of error rates. This was unexpected because the data was complex and did not show clear patterns, which usually makes Neural Networks the better choice since they're good at finding hidden details. We thought Random Forest is more suitable for sorting things into categories, but it did considerably well at predicting numbers in our tests. It managed the tricky data well and didn't need much tuning to perform well, which suggests that Random Forests might be better at handling complicated data than we thought. However, the Neural Network did not do as well as we hoped. It might have learned too much unnecessary detail from the training data, making it less accurate on new data. It was also quite sensitive to how we prepared the data, like scaling and using PCA. The latter one had a surprisingly positive effect.

Furthermore, it is very interesting that our neural network loss value did not improve by increasing the training epochs from 10 to 3000. This makes it possible store our model in a small file, whereas the Random Tree Regressor model's size was exceeding the limit of 50 MB by a small amount. For the notebook we therefore decreased the tuned metrics, which did not affect much the loss.

Because of these findings, it might be good to try out Linear Regression. At first, we did not think it would be suitable since the data does not follow a straightforward pattern. But given that the more complex models did not do as well as expected, a simpler model like Linear Regression might actually do a better job. These results show us that choosing the right model for the data is important.

The most interesting thing we observed was that the scaler we used in our training script and also initially applied in the *get_score* function excessively increased the MSE values while training and validation. We decided to not scale our data for the final model because we saw that especially the neural network did much better without scaler and RF was not affected that much. We reflectively integrated PCA after DOC and just before splitting the data. It would be interesting to see what further reduction could affect the performance of

our model. There were multiple things that we just noticed after the experimenting phase and due to time limits could not include into our script, for instance early stopping in the neural network to prevent overfitting.

V. CONCLUSION

Model	Training Dataset MAE	Hidden Test Dataset MAE
Random Forest	4427.54	4184.90
Neural Network	28399.40	28651.19

TABLE II

PERFORMANCE OF RANDOM FOREST AND NEURAL NETWORK MODELS
ON THE TRAINING AND HIDDEN TEST DATASETS

The performance of the Random Forest and Neural Network models was evaluated on both the training and also hidden test datasets which indicates how well our models can generalize on unknown data. The Mean Absolute Error (MAE) was used as the evaluation metric. The results are summarized in the table above II. In accordance to our training data the RF regressor outperformed the neural network but overall our score is disappointing, considering that a simple linear regression model would do better than our models. Before the final score, which we achieved by using unscaled and PCA unreduced data we even reached slightly better performance which highlights the importance of data preprocessing. Though we cannot really comprehend that score, the learning process while tuning our models will help us for further ML tasks.